Analysis of CPU vs. GPU performance in real-time image processing

Introduction

The real-time image processing phenomenon plays a significant role in modern visual computing applications like augmented reality, computer vision, and live video enhancement. By achieving high frame rates (FPS) which is essential to ensure the smooth and responsive user experience. However, performance can vary significantly depending on factors like execution hardware (CPU vs. GPU), image resolution, filtering algorithms, compiler mode (Debug vs. Release). Geometric transformations that are applied to the frames can also affect performance in a crucial way.

This study's goal is to answer the following research question:

„How do hardware settings, image resolution, filters and their respective complexity and geometric transformations influence the real-time performance of image processing pipelines, and to what extent can GPU parallelization improve execution speed compared to CPU processing?"

The study hypothesizes the following:

1. **GPU execution will consistently outperform CPU execution**, especially for filters that are computationally intensive/expensive.
2. **Higher resolutions will lead to lower FPS**.
3. **Release builds will show higher performance than Debug builds** thanks to compiler optimizations.
4. **Interactive geometric transformations will reduce FPS**, but the reduction is going to be less extensive on GPUs.

Experimental Design and Methodology

Hardware and Software Environment

- **CPU**: *AMD Ryzen 7 7735HS*
- **GPU**: *NVIDIA GeForce RTX 4050 Laptop GPU*
- **Memory**: *16 GB RAM*
- **Development environment**: *2022 Visual Studio IDE, Windows SDK – Windows 10.0.26100.1*
- **External libraries**: opencv:x64-windows, glfw3:x64-windows, glad:x64-windows, glm:x64-windows

Test Dataset

- Input: Live webcam feed
- Frame rates measured over a 20-second capture period for each configuration

Experimental Variables

Variable Category	Conditions Tested
Resolution	Standard HD (1280×720) and Low (640×360)
Filters	Pixelate, SinCity, Comic
Execution Hardware	CPU vs. GPU
Build Configuration	Debug vs. Release
Geometric Transformations	None vs. Real-time rotation/pan/zoom

Each test scenario consisted of running the same processing pipeline under each combination of conditions and recording average FPS.

System design

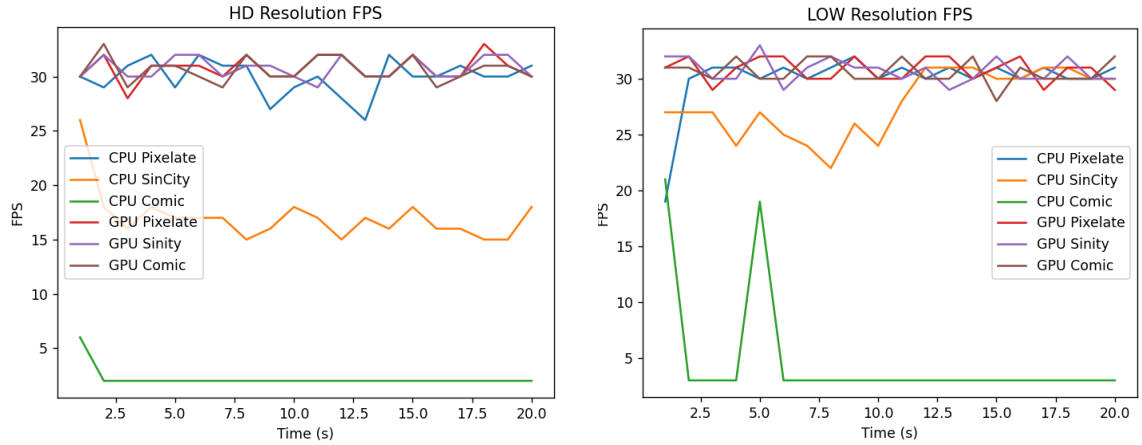
The created application's architecture is a simple MV architecture, with and additional services component that interacts with the models, and is used by the views. The views contain logic for rendering and compiling shaders, showing windows, handling user input and showing visual results for the user based on their input. The user can input information for transformation by using the mouse and for filters by pressing keys while the main window is in the focus. The services provide business logic for file reading, logging, matrix computations, transform computations. The models contain data for filters and app-wide constants.

Results

CPU vs. GPU Performance

Filter	CPU FPS (HD)	GPU FPS (HD)	CPU FPS (Low)	GPU FPS (Low)
Pixelate	29.95	30.75	30	30.8

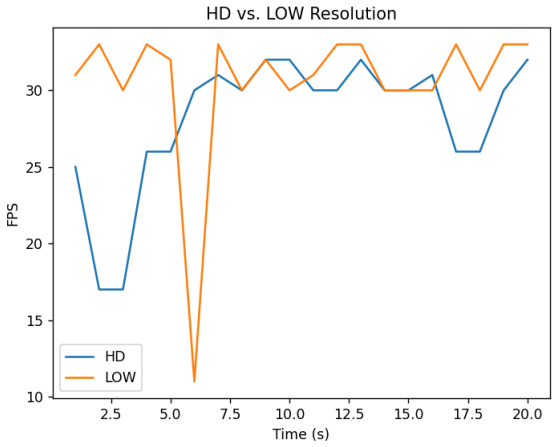
Filter	CPU FPS (HD)	GPU FPS (HD)	CPU FPS (Low)	GPU FPS (Low)
SinCity	17.05	30.75	27.8	30.75
Comic	2.2	30.6	4.7	30.65



Observation: The GPU consistently achieved higher FPS across the filters, particularly for the Comic and SinCity filters, which involve more operations to calculate.

Filters: Normal – Pixelate – SinCity – Comic

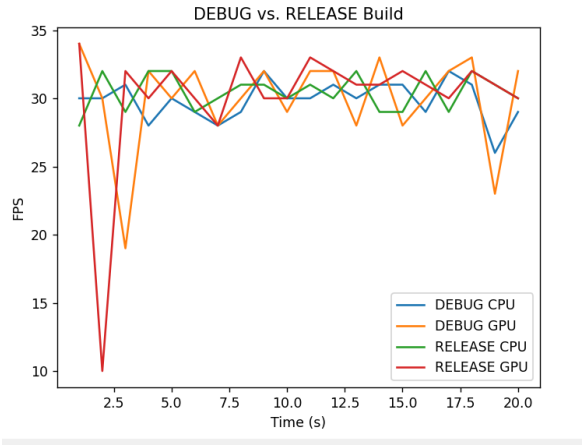
Resolution Comparison



Observation: The CPU performance dropped more at higher resolutions, while the GPU maintained higher stability throughout the testing.

Debug vs. Release Build

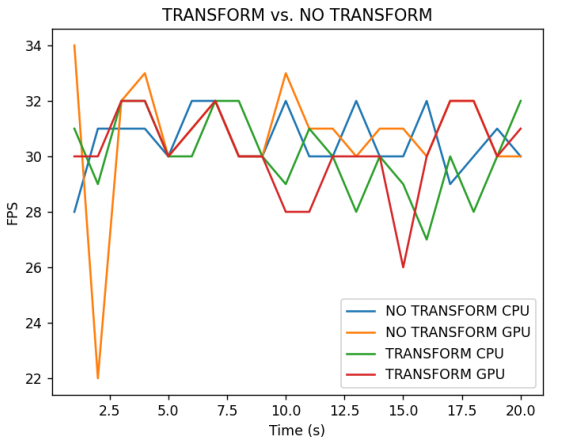
Hardware	Debug FPS (HD)	Release FPS (HD)
CPU	29.85	30.45
GPU	29.95	30.1



Observation: Release builds have improved FPS due to compiler optimizations, instruction scheduling and loop unrolling.

3.4 Effect of Geometric Transformations

Execution Mode	No Transform (FPS)	Interactive Transform (FPS)
CPU	30.55	*30.1*
GPU	30.75	*30.2*



Observation: Transform operations introduced an additional computational load, reducing FPS. The GPU remained more resilient due to parallel execution of pixel and vertex operations.

Discussion

The results support the hypothesis that the GPU provides a better performance advantage in real-time image processing. Due to the fact that GPUs' design enables them for parallel processing of large blocks of pixel data. They scale better when resolution and filter complexity increase. In contrast to the CPU, which showed a close to linear performance drop as frame size and computation increased. It is consistent with the CPU's emphasis on sequential processing.

The difference between Debug and Release builds shows the importance of compiler optimizations. Without them, unnecessary overhead can limit the performance and may imply inefficiency in the algorithm design.

Moreover, geometric transformations reduced performance on both processing units, but the GPU could maintain higher FPS. This was expected because newer GPUs contain dedicated hardware pipelines for vertex and fragment transformations, whereas CPUs have to compute transformations per pixel through the general-purpose arithmetic.

The implications for real-time visual computing are clear:

- **GPU acceleration is necessary for interactive or high-resolution applications.**

- **Algorithm and resolution choices need to be matched in order to target performance needs.**
- **Optimization settings (Release build, parallel processing) highly impact real-world usability of the application**

The implementation uses OpenCV to capture live camera frames, then uploads them as a texture and then applies visual effects using the CPU-side image processing or the GPU-side shader operations. The rendering is done through a full screen quad in OpenGL, with UV coordinates passed to the fragment shader. A uniform 3×3 transform matrix allows interactive pan, zoom and rotation in the texture space, in order to avoid expensive CPU-side operations when GPU mode is active.

The shader design uses the uFilterType flag to select between effects such as Pixelation, Sin City and Comic. Each filter is implemented compactly in GLSL and operates per-fragment in normalized UV space. The Pixelation filter uses a block-based sampling strategy. The Sin City filter converts non-red areas to grayscale using a specific luminance formula. The Comic filter effect relies on „fwidth" for edge detection and uses colour quantization.

Limitations of the system and shaders

- **Single-pass fragment operations:** All effects are limited to numerous local image-based operations. More advanced filters (e.g. blur) would require additional passes or framebuffers.
- **Fixed transform precision:** Transforming UV coordinates in the fragment shader works well for basic motion, but more complex lens effects would need higher-order transforms.
- **No colour management or gamma correction:** Colour values are treated linearly, but it is possible to achieve potential accuracy improvements from proper colour-space handling.
- **CPU/GPU mode split introduces duplication:** The CPU mode re-applies transformations and filters redundantly, which can lead to performance drops on weaker hardware.
- **Edge detection in Comic filter is resolution-dependent:** The appearance of the ink outlines changes with texture resolution and view scale because „fwidth" is tied to screen-space derivatives.

Conclusion

This study demonstrates, that GPU-based processing is much more effective than just CPU processing for real-time image processing tasks, especially at higher resolutions and with using computationally expensive filters. Performance could be further improved when running in Release mode instead of Debug mode and when geometric transformations usage is minimized or processed through the GPU.